

Single Dimensional Array:**1.Aim:**

To write a C++ program to find the **smallest** and **largest** elements in a single-dimensional array.

Algorithm:

Step 1: Start.

Step 2: Read the number of elements n.

Step 3: Read the array elements.

Step 4: Initialize smallest and largest with the first element.

Step 5: Compare each element with smallest and largest.

Step 6: Update smallest if a smaller element is found.

Step 7: Update largest if a larger element is found.

Step 8: Display the smallest and largest elements.

Step 9: Stop.

Program Code:

```
#include<iostream>
using namespace std;
int main(){
    int n;
    cout<<"Enter the no.of elements:";
    cin>>n;
    int arr[n];
    cout<<"Enter the array elements:";
    for(int i=0;i<n;i++){
        cin>>arr[i];
    }
    int smallest=arr[0];
    int largest=arr[0];
    for(int i=1;i<n;i++){
        if(arr[i]<smallest){
            smallest=arr[i];
        }
        if(arr[i]>largest){
            largest=arr[i];
        }
    }
    cout<<"Smallest Element:"<<smallest<<endl;
    cout<<"Largest Element:"<<largest<<endl;
    return 0;
}
```

Output:

```
Enter the no.of elements:5
Enter the array elements:1 9 5 7 3
Smallest Element:1
Largest Element:9

=== Code Execution Successful ===
```

Result:

The C++ program to find the **smallest** and **largest** elements in a single-dimensional array was successfully executed, and the desired output was obtained.

2.Aim:

To write a C++ program to merge two single-dimensional arrays into a single array.

Algorithm:

Step 1: Start.

Step 2: Read the sizes of the two arrays.

Step 3: Read the elements of the first array.

Step 4: Read the elements of the second array.

Step 5: Copy all elements of the first array into the merged array.

Step 6: Copy all elements of the second array into the merged array.

Step 7: Display the merged array.

Step 8: Stop.

Program Code:

```
#include<iostream>
using namespace std;
int main(){
    int n1,n2;
    cout<<"Enter the size of the first array:";
    cin>>n1;
    int arr1[n1];
    cout<<"Enter the size of the second array:";
    cin>>n2;
    int arr2[n2];
    cout<<"Enter the first array elements:";
    for(int i=0;i<n1;i++){
        cin>>arr1[i];
    }
    cout<<"Enter the second array elements:";
    for(int i=0;i<n2;i++){
        cin>>arr2[i];
    }
    int merge[n1+n2];
    for(int i = 0; i < n1; i++) {
        merge[i] = arr1[i];
    }
    for(int i = 0; i < n2; i++) {
        merge[n1 + i] = arr2[i];
    }
    cout << "Merged array: ";
    for(int i = 0; i < n1 + n2; i++) {
        cout << merge[i] << " ";
    }
    return 0;
}
```

Output:

```
Enter the size of the first array:3
Enter the size of the second array:3
Enter the first array elements1 2 3
Enter the second array elements4 5 6
Merged array: 1 2 3 4 5 6
```

```
=== Code Execution Successful ===
```

Result:

The C++ program to merge two single-dimensional arrays was successfully executed, and the merged array was displayed correctly.

3.Aim:

To write a C++ program to find the **sum of all elements** in a single-dimensional array.

Algorithm:

- Step 1: Start.
- Step 2: Read the number of elements n.
- Step 3: Read the array elements.
- Step 4: Initialize sum = 0.
- Step 5: Add each array element to sum.
- Step 6: Display the sum of all elements.
- Step 7: Stop.

Program Code:

```
#include<iostream>
using namespace std;
int main(){
    int n,sum=0;
    cout<<"Enter the no.of elements:";
    cin>>n;
    int arr[n];
    cout<<"Enter the array elements:";
    for(int i=0;i<n;i++){
        cin>>arr[i];
    }
    for(int i=0;i<n;i++){
        sum+=arr[i];
    }
    cout<<"Sum="<<sum;
    return 0;
}
```

Output:

```
Enter the no.of elements:5
Enter the array elements:1 2 3 4 5
Sum=15

=== Code Execution Successful ===
```

Result:

The C++ program to find the **sum of all elements** in a single-dimensional array was successfully executed, and the correct sum was displayed.

Multi Dimensional Array:**1.Aim:**

To write a C++ program to read and display the elements of a 2D array.

Algorithm:

- Step 1: Start.
- Step 2: Read the number of rows and columns.
- Step 3: Read the elements of the 2D array.
- Step 4: Display the elements row by row.
- Step 5: Stop.

Program code:

```
#include <iostream>
using namespace std;
int main() {
    int r, c;
    cout << "Enter rows and columns: ";
    cin >> r >> c;
    int arr[r][c];
    cout << "Enter the elements:";
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            cin >> arr[i][j];
        }
    }
    cout << "2D Array:\n";
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            cout << arr[i][j] << " ";
        }
    }
    return 0;
}
```

EX-2

Date:

Poorna sri

953625148038

Output:

```
Enter rows and columns: 3 3
Enter the elements:1 2 3 4 5 6 7 8 9
2D Array:
1 2 3
4 5 6
7 8 9

=== Code Execution Successful ===
```

Result:

The C++ program to read and display the elements of a 2D array was successfully executed, and the desired output was obtained.

2. Aim:

To write a C++ program to find the transpose of a matrix using a 2d array.

Algorithm:

Step 1: Start.

Step 2: Read the number of rows and columns.

Step 3: Read the matrix elements.

Step 4: Interchange rows and columns to form the transpose.

Step 5: Display the transpose matrix.

Step 6: Stop.

Program Code:

```
#include <iostream>
using namespace std;
int main() {
    int r, c;
    cout << "Enter rows and columns: ";
    cin >> r >> c;
    int arr[r][c];
    cout << "Enter the elements:";
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            cin >> arr[i][j];
        }
    }
    cout << "Transpose Matrix:\n";
    for (int i = 0; i < c; i++) {
        for (int j = 0; j < r; j++) {
            cout << arr[j][i] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

EX-2

Date:

Poorna sri

953625148038

Output:

```
Enter rows and columns: 3 3
Enter the elements:1 2 3 4 5 6 7 8 9
Transpose Matrix:
1 4 7
2 5 8
3 6 9

=== Code Execution Successful ===
```

Result:

The C++ program to find the transpose of a matrix using a 2D array was successfully executed, and the transpose matrix was displayed correctly.

3. Aim:

To write a C++ program to reverse the elements of a 2D array.

Algorithm

Step 1: Start.

Step 2: Read the number of rows and columns.

Step 3: Read the elements of the 2D array.

Step 4: Reverse the elements of the array.

Step 5: Display the reversed 2D array.

Step 6: Stop.

Program Code:

```
#include <iostream>
using namespace std;
int main() {
    int r, c;
    cout << "Enter rows and columns: ";
    cin >> r >> c;
    int arr[r][c];
    cout << "Enter the elements:";
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            cin >> arr[i][j];
        }
    }
    cout << "Reversed Matrix:\n";
    for (int i = r-1; i >= 0; i--) {
        for (int j = c-1; j >= 0; j--) {
            cout << arr[j][i] << " ";
        }
        cout << endl;
    }
    return 0;
}
```

Output:

```
Enter rows and columns: 3 3
Enter the elements:1 2 3 4 5 6 7 8 9
2D Array:
1 2 3
4 5 6
7 8 9

=== Code Execution Successful ===
```

Result:

The C++ program to reverse the elements of a 2D array was successfully executed, and the reversed array was displayed correctly.

EX-2
Date:

Poorna sri
953625148038